

Laporan Milestone 4 Intermediate Code & Interpreter

IF2224 TEORI BAHASA FORMAL DAN OTOMATA



Disusun oleh:

BST - brawlstars

Kevin Wirya Valerian 13524019

Ray Owen Martin 13524033

Emilio Justin 13524043

Philipp Hamara 13524101

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

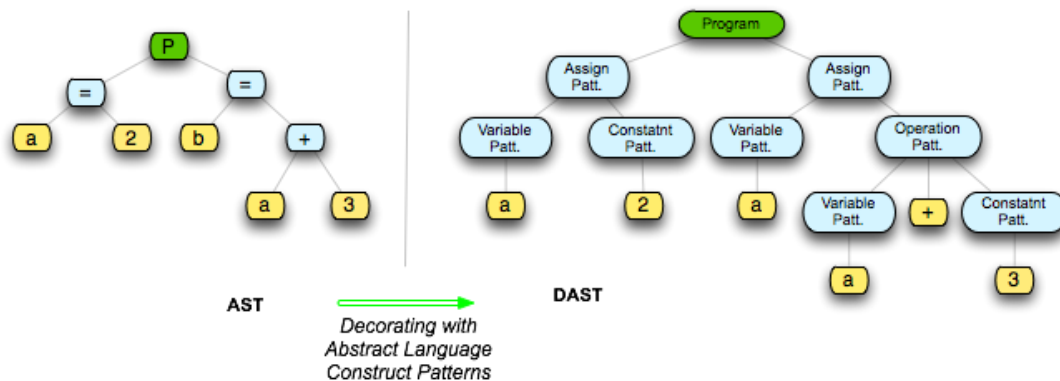
Daftar Isi

Daftar Isi.....	2
I. Teori Singkat.....	3
A. Intermediate Code.....	3
a. Three-Adress Code.....	3
b. Translasi Control Flow.....	4
B. Arsitektur Mesin Eksekusi.....	4
a. Virtual Machine dan Siklus Eksekusi.....	4
b. Memori Eksekusi: Stack.....	5
c. Runtime Type Checking.....	5
C. Vulnerabilities pada Interpreter.....	5
a. Kerentanan Memori dan Akses: Out-of-Bounds Variable Access.....	5
b. Out-of-Bounds Variable Access.....	6
II. Perancangan & Implementasi.....	8
A. Perancangan.....	8
B. Implementasi.....	8
III. Pengujian.....	9
A. Pengujian 1.....	9
B. Pengujian 2.....	9
C. Pengujian 3.....	15
D. Pengujian 4.....	17
E. Pengujian 5.....	18
F. Pengujian 6.....	20
G. Pengujian 7.....	21
H. Pengujian 8.....	21
I. Pengujian 9.....	22
J. Pengujian 10.....	24
K. Pengujian 11.....	25
L. Pengujian 12.....	26
M. Pengujian 13.....	27
N. Pengujian 14.....	29
O. Pengujian 15.....	30
IV. Kesimpulan & Saran.....	31
A. Kesimpulan.....	31
B. Saran.....	31
Referensi.....	32
Lampiran.....	33

I. Teori Singkat

A. Intermediate Code

Tujuan utama dari Intermediate Code Generation adalah transformasi Decorated AST menjadi instruksi operasional standar, yang disebut sebagai Intermediate Code, yang disebut juga sebagai sintesis. Decorated AST menjadi syarat utama karena menjadi input untuk proses sintesis. Decorated AST dipilih secara khusus karena mengandung lebih banyak informasi dibandingkan AST dan Parse Tree. Alasannya karena bagian Back-end (yang dilakukan di milestone ini) sudah tidak memperdulikan sintaks, justru membutuhkan informasi semantik seperti tipe data dan referensi tabel simbol.



Gambar 1. Ilustrasi Perbandingan AST biasa dengan *Decorated AST*
(Sumber: https://epl.di.uminho.pt/~gepl/GEPL_DS/Alma2/dapast.php)

a. Three-Adress Code

Proses transformasi decorated AST menjadi Intermediate Code umumnya menggunakan format Three-Address Code (TAC) sebagai outputnya. Format ini menghasilkan desain tiap baris instruksi yang sangat sederhana dengan maksimal 3 alamat memori, yang terdiri dari 2 operand sumber dan 1 lokasi tujuan.

Untuk memungkinkan proses CPU melakukan perhitungan matematis yang kompleks dan panjang, diperlukan pemecahan menjadi operasi biner sederhana. Batasan ini divisualisasikan dengan penggunaan variabel sementara, sementara proses pemecahannya sudah diatur di bagian parser ketika membuat AST.

b. Translasi Control Flow

Pola ekspresi matematika yang tetap mendukung pembuatan TAC yang relatif mudah untuk bagian tersebut. Yang menjadi tantangan sulit adalah transformasi control flow, seperti if-else dan looping. Menghadapi tantangan ini, interpreter umumnya menggunakan Label dan Jump (Goto).

1. Label

ditandai dengan prefix `L_`, label berfungsi sebagai penanda statis posisi baris dalam TAC. Contoh: `L_START`, `L_ELSE`, `L_END`.

2. Jump

menggeser baris yang sedang dieksekusi ke label tertentu. Dapat dikombinasikan dengan penggunaan conditional seperti `IF_FALSE`. Jump tidak mewajibkan penggeseran harus dilakukan ke baris setelahnya, melainkan juga boleh ke baris sebelumnya, sehingga mendukung logika dan proses looping.

B. Arsitektur Mesin Eksekusi

Setelah fase sintesis untuk mendapatkan Intermediate Code, fase eksekusi harus dilakukan untuk melengkapi tugas ini. Fase ini diemban oleh Interpreter layaknya suatu Virtual Machine.

a. Virtual Machine dan Siklus Eksekusi

Selayaknya CPU, Interpreter membaca instruksi baris per baris. Pelacak baris yang sedang dieksekusi disebut Instruction Pointer (IP) atau Program Counter (PC).

Siklus eksekusi yang diikuti interpreter terdiri dari 3 tahap utama:

1. Fetch

membaca baris instruksi TAC yang sedang ditunjuk IP

2. Decode

memahami makna perintah baris instruksi yang di-fetch

3. Execute

melakukan aksi nyata seperti hitung nilai hasil operasi matematika

Setiap kali tahap execute selesai, IP akan di-increment untuk lanjut ke baris selanjutnya kecuali instruksi `GOTO`.

b. Memori Eksekusi: Stack

Penyimpanan variabel memerlukan bagian vital dalam arsitektur interpreter: stack. Prinsip Last In First Out (LIFO) pada Stack mendukung kelancaran alur program ketika dihadapkan dengan blok kode bersarang. Alasan kuat lain untuk pemilihan stack sebagai representasi memori adalah untuk isolasi ruang lingkup variabel, termasuk bagian penting stack yang direserve dan ditempatkan di bagian paling dalam stack.

Representasi ini menghasilkan stack frame setiap kali interpreter membaca blok kode baru atau memanggil suatu fungsi. Frame ini berisi semua variabel lokal, argumen yang di-pass dan return address. Pemanggilan fungsi akan push Stack Frame terbaru yang dibuat ke bagian top dari stack. Ketika fungsi selesai dieksekusi barulah sisa frame akan di-pop dari stack memory. Selain mencegah variabel lokal suatu fungsi diakses fungsi lain, representasi ini juga menghemat penggunaan memori.

c. Runtime Type Checking

Tipe data yang sudah dicek validitasnya di bagian semantic analysis tetap perlu diketahui oleh interpreter. Ini penting untuk bahasa pemrograman yang mendukung perubahan tipe data variabel saat runtime atau deklarasi tipe data implisit, sehingga tiap proses eksekusi TAC (yaitu saat runtime), interpreter harus melakukan cek tipe data untuk mencegah kerusakan berlebih pada komputer. Proses cek akan dilakukan untuk tipe data dari nilai aktual pada stack memori, sementara kesalahan tipe data akan menghentikan eksekusi dan melempar runtime error.

Pada tugas ini, bagian ini tidak wajib dilakukan, sehingga tidak diimplementasikan.

C. Vulnerabilities pada Interpreter**a. Kerentanan Memori dan Akses: Out-of-Bounds Variable Access**

Ibarat exception yang mencegah kerusakan berlebih, vulnerabilities pada bahasa pemrograman juga harus memiliki mekanisme perlindungan internal. Jenis-jenis kerentanan ini adalah sebagai berikut:

1. Kerentanan pada Stack

Stack sebagai representasi memori menjadi jiwa dari eksekusi interpreter. Banyak kerentanan yang dapat muncul seperti:

a. Stack Overflow

Ketika program menjalankan Push Stack Frame yang terlalu banyak, infinite recursion dapat terpicu sehingga menyebabkan program berjalan secara tidak masuk akal. Untuk mencegah hal ini, dapat diimplementasikan throw error ketika frame yang dipush melebihi batas kapasitas hardware yang dialokasikan interpreter.

b. Stack Underflow

Stack memori yang memiliki informasi penting di bagian paling dalam stack dapat menjadi rentan ketika pop dilakukan berlebih, apalagi jika mencapai base dari stack.

c. Stack Smashing

Posisi return address pada stack memori yang bersebelahan dengan memori lokal berisiko menyebabkan data variabel lokal menimpa return address. Ini menjadi permasalahan klasik security karena return address yang ditimpa bisa mengarah ke kode berbahaya.

d. Stack Corruption

Keseimbangan jumlah push dan pop yang dilakukan pada stack adalah sesuatu yang mutlak harus dipertahankan. Jika jumlah operasi pop lebih banyak, nilai yang di-pop bisa jadi tidak berasal dari blok kode yang seharusnya. Sebaliknya jika jumlah operasi push lebih banyak, maka akan ada "sampah" memori tertinggal yang merusak pembacaan variabel selanjutnya.

b. Out-of-Bounds Variable Access

Kerentanan ini adalah kerentanan umum yang sering terjadi akibat kesalahan programmer. Tiap akses indeks harus divalidasi oleh interpreter. Ketika indeks yang diakses tidak sesuai, interpreter harus mencegatnya dan melempar exception.

c. Invalid Jump Targets

Ketika TAC meminta melompat ke label tertentu tapi label tersebut tidak pernah ada di daftar instruksi atau TAC meminta melompat ke tengah instruksi yang bukan batas operasi, interpreter perlu memverifikasi target Jump sebelum atau ketika dieksekusi agar IP tidak tersesat.

d. Numerical Overflow/Underflow

Tipe data di dalam komputer memiliki kapasitas maksimum berdasarkan alokasi memori yang diberikan. Interpreter harus mendeteksi batasan ini untuk mencegah perhitungan dan logika perulangan. Jika di bawah batas, maka program dihentikan dengan `UnderflowError`, jika di atas, maka `OverflowError`.

II. Perancangan & Implementasi

A. Perancangan

1. Intermediate Code Generator

Intermediate Code Generator menerima Decorated AST dan Symbol Table, lalu menurunkannya menjadi instruksi Three-Address Code berbasis Stack Machine.

- Struktur instruksi memakai ICIInstr (opcode, level, operand, dan string operand). Ini memungkinkan format instruksi yang sama untuk angka maupun string, sehingga write dan writeln dapat mencetak literal string tanpa membuat instruksi khusus
- Strategi pembuatan kode menggunakan traversal AST dan teknik patching label (JMP/JPC) untuk kontrol alur IF, WHILE, REPEAT, dan CASE
- Operasi matematika atau logika diubah menjadi urutan instruksi LIT (simpan nilai konstan), LOD (ambil nilai variabel), dan OPR (lakukan operasi) menggunakan mekanisme *stack* (tumpukan). Ini membuat eksekusi lebih mudah karena hanya perlu operasi *push* (masukkan) dan *pop* (keluarkan).

2. Interpreter (Virtual Machine)

Interpreter mengeksekusi daftar instruksi ICIInstr dengan model stack machine.

- Memori runtime memakai stack tunggal. Setiap kali sebuah fungsi dipanggil, sistem membuat ruang memori (*stack frame*) yang berisi SL (static link), DL (dynamic link), dan RA (return address)
- Value menyimpan tiga tipe runtime (Number, String, Char) agar output write/writeln dan operasi string dapat ditangani tanpa casting manual
- Interpreter menambahkan runtime checks sesuai bonus: invalid jump, stack overflow/underflow, division by zero, dan bounds checking untuk array
- Informasi tipe dari Symbol Table dipetakan ke alamat untuk menjaga konsistensi tipe saat load/store

B. Implementasi

1. Fungsi/Kelas Utama

- `IntermediateCodeGen::generate()` membentuk kode utama dan memisahkan subprogram agar ada loncatan awal ke main (entry point)
- `IntermediateCodeGen::genStmt()` dan `genExpr()` menangani penurunan AST ke instruksi stack
- `Interpreter::execute()` menjalankan loop fetch-decode-execute
- `Interpreter::execInstruction()` menangani semua opcode termasuk LIT/LOD/STO, CAL/RET, serta OPR
- `Interpreter::base()` dipakai untuk traversing static link agar scope akses variabel lintas level tetap benar

2. Justifikasi Implementasi

- Pencatatan subprogram menggunakan `subprogStartLine` membuat CAL bisa langsung melompat ke label yang tepat tanpa dua-pass compiler kompleks
- OPR dipakai sebagai satu pintu untuk operasi aritmatika/logika serta `write/writeln` agar format instruksi tetap ringkas dan eksekusi tetap efisien

3. Implementasi Program

Implementasi Intermediate Code Generation dilakukan dengan memetakan setiap node AST ke instruksi stack yang relevan:

- Assign → evaluasi ekspresi, lalu STO (atau indirect store untuk array/record)
- If/While/Repeat/For/Case → kombinasi JPC dan JMP dengan patching label
- Function/Procedure Call → CAL dengan penyesuaian level lexical, argumen didorong ke stack terlebih dahulu
- Array/Record Access → offset dihitung oleh `pushVarAddress()` dan dijaga dengan bounds check (OPR 22)
- Interpreter mengeksekusi instruksi dengan stack frame per subprogram. CAL akan menyisipkan SL/DL/RA, sementara RET mengembalikan kontrol dan nilai return (jika fungsi)

III. Pengujian

A. Pengujian 1

Input

```
program Hello;

var
  a, b: integer;

begin
  a := 5;
  b := a + 10;
  writeln('Result = ', b);
end.
```

Output

```
=== INTERMEDIATE CODE ===
0 INT 0 5
1 LIT 0 5
2 STO 0 3
3 LOD 0 3
4 LIT 0 10
5 OPR 0 2
6 STO 0 4
7 LIT 0 'Result = '
8 OPR 0 13
9 LOD 0 4
10 OPR 0 14
11 RET

=== INTERPRETER OUTPUT ===
Result = 15
```

B. Pengujian 2

Input

```
program Everything;

var
  i, j: integer;
  x: real;
  ch: char;
  e: (A, B, C);
  r: 1..3;
  mat: array[1..3] of array[1..3] of integer;
  rec: record
```

```
    age: integer;
    code: integer
end;

procedure Update(a, b: integer; v: array[1..3] of integer);
begin
    v[1] := a + b;
    v[2] := (a * 2) div 2;
    v[3] := (v[1] + v[2]) mod 3;
end;

function NextVal(n: integer): integer;
begin
    NextVal := n + 1;
end;

begin
    i := 1;
    j := -i + 2 * 3 div 2 mod 2;
    x := 6 / 2;
    ch := 'z';
    mat[1,2] := j;
    rec.age := mat[1][2];
    rec.code := NextVal(rec.age);

    if not (j <= 0) or (i <> 0) then
        j := j + 1
    else
        j := j - 1;

    case j of
        1: j := 10;
        2, 3: j := 20;
    end;

    while (j > 0) and (i < 10) do
        begin
            j := j - 1;
        end;

    repeat
        j := j + 1;
    until j >= 2;

    for i := 1 to 3 do
        begin
            Update(i, j, mat[1]);
        end;

    for i := 3 downto 1 do
        begin
            j := j + i;
        end;
```

```
Update(i, j, mat[2]);  
writeln('done', j, x, ch);  
end.
```

Output

```
=== INTERMEDIATE CODE ===  
0 JMP 0 65  
1 INT 0 6  
2 LOD 0 3  
3 LOD 0 4  
4 OPR 0 2  
5 LIT 0 5  
6 LIT 0 1  
7 LIT 0 1  
8 LIT 0 3  
9 OPR 0 22  
10 LIT 0 1  
11 OPR 0 3  
12 OPR 0 2  
13 OPR 0 19  
14 LOD 0 3  
15 LIT 0 2  
16 OPR 0 4  
17 LIT 0 2  
18 OPR 0 5  
19 LIT 0 5  
20 LIT 0 2  
21 LIT 0 1  
22 LIT 0 3  
23 OPR 0 22  
24 LIT 0 1  
25 OPR 0 3  
26 OPR 0 2  
27 OPR 0 19  
28 LIT 0 5  
29 LIT 0 1  
30 LIT 0 1  
31 LIT 0 3  
32 OPR 0 22  
33 LIT 0 1  
34 OPR 0 3  
35 OPR 0 2  
36 OPR 0 18  
37 LIT 0 5  
38 LIT 0 2  
39 LIT 0 1  
40 LIT 0 3  
41 OPR 0 22  
42 LIT 0 1  
43 OPR 0 3
```

```
44 OPR 0 2
45 OPR 0 18
46 OPR 0 2
47 LIT 0 3
48 OPR 0 6
49 LIT 0 5
50 LIT 0 3
51 LIT 0 1
52 LIT 0 3
53 OPR 0 22
54 LIT 0 1
55 OPR 0 3
56 OPR 0 2
57 OPR 0 19
58 RET
59 INT 0 4
60 LOD 0 3
61 LIT 0 1
62 OPR 0 2
63 STO 0 0
64 RET
65 INT 0 13
66 LIT 0 1
67 STO 0 3
68 LOD 0 3
69 OPR 0 1
70 LIT 0 2
71 LIT 0 3
72 OPR 0 4
73 LIT 0 2
74 OPR 0 5
75 LIT 0 2
76 OPR 0 6
77 OPR 0 2
78 STO 0 4
79 LIT 0 6
80 LIT 0 2
81 OPR 0 5
82 STO 0 5
83 LIT 0 122
84 STO 0 6
85 LOD 0 4
86 LIT 0 9
87 LIT 0 1
88 LIT 0 1
89 LIT 0 3
90 OPR 0 22
91 LIT 0 1
92 OPR 0 3
93 OPR 0 2
94 LIT 0 2
95 LIT 0 1
96 LIT 0 3
```

```
97 OPR 0 22
98 LIT 0 1
99 OPR 0 3
100 OPR 0 2
101 OPR 0 19
102 LIT 0 9
103 LIT 0 1
104 LIT 0 1
105 LIT 0 3
106 OPR 0 22
107 LIT 0 1
108 OPR 0 3
109 OPR 0 2
110 LIT 0 2
111 LIT 0 1
112 LIT 0 3
113 OPR 0 22
114 LIT 0 1
115 OPR 0 3
116 OPR 0 2
117 OPR 0 18
118 LIT 0 12
119 LIT 0 0
120 OPR 0 2
121 OPR 0 19
122 LIT 0 12
123 LIT 0 0
124 OPR 0 2
125 OPR 0 18
126 CAL 0 59
127 LIT 0 12
128 LIT 0 0
129 OPR 0 2
130 OPR 0 19
131 LOD 0 4
132 LIT 0 0
133 OPR 0 12
134 OPR 0 17
135 LOD 0 3
136 LIT 0 0
137 OPR 0 8
138 OPR 0 16
139 JPC 0 145
140 LOD 0 4
141 LIT 0 1
142 OPR 0 2
143 STO 0 4
144 JMP 0 149
145 LOD 0 4
146 LIT 0 1
147 OPR 0 3
148 STO 0 4
149 LOD 0 4
```

```
150 LIT 0 1
151 OPR 0 7
152 JPC 0 156
153 LIT 0 10
154 STO 0 4
155 JMP 0 156
156 LOD 0 4
157 LIT 0 0
158 OPR 0 11
159 LOD 0 3
160 LIT 0 10
161 OPR 0 9
162 OPR 0 15
163 JPC 0 169
164 LOD 0 4
165 LIT 0 1
166 OPR 0 3
167 STO 0 4
168 JMP 0 156
169 LOD 0 4
170 LIT 0 1
171 OPR 0 2
172 STO 0 4
173 LOD 0 4
174 LIT 0 2
175 OPR 0 10
176 JPC 0 169
177 LIT 0 1
178 STO 0 3
179 LOD 0 3
180 LIT 0 3
181 OPR 0 12
182 JPC 0 200
183 LOD 0 3
184 LOD 0 4
185 LIT 0 9
186 LIT 0 1
187 LIT 0 1
188 LIT 0 3
189 OPR 0 22
190 LIT 0 1
191 OPR 0 3
192 OPR 0 2
193 OPR 0 18
194 CAL 0 1
195 LOD 0 3
196 LIT 0 1
197 OPR 0 2
198 STO 0 3
199 JMP 0 179
200 LIT 0 3
201 STO 0 3
202 LOD 0 3
```



```
203 LIT 0 1
204 OPR 0 10
205 JPC 0 215
206 LOD 0 4
207 LOD 0 3
208 OPR 0 2
209 STO 0 4
210 LOD 0 3
211 LIT 0 1
212 OPR 0 3
213 STO 0 3
214 JMP 0 202
215 LOD 0 3
216 LOD 0 4
217 LIT 0 9
218 LIT 0 2
219 LIT 0 1
220 LIT 0 3
221 OPR 0 22
222 LIT 0 1
223 OPR 0 3
224 OPR 0 2
225 OPR 0 18
226 CAL 0 1
227 LIT 0 'done'
228 OPR 0 13
229 LOD 0 4
230 OPR 0 13
231 LOD 0 5
232 OPR 0 13
233 LOD 0 6
234 OPR 0 14
235 RET

=== INTERPRETER OUTPUT ===
done83z
```

C. Pengujian 3

Input

```
program TestArr;

var
  a: array[1..5] of integer;
  i: integer;

begin
  for i := 1 to 5 do
    a[i] := i * 10;
  for i := 1 to 5 do
    writeln(a[i]);
```

end.

Output

```
=== INTERMEDIATE CODE ===  
0 INT 0 9  
1 LIT 0 1  
2 STO 0 8  
3 LOD 0 8  
4 LIT 0 5  
5 OPR 0 12  
6 JPC 0 24  
7 LOD 0 8  
8 LIT 0 10  
9 OPR 0 4  
10 LIT 0 3  
11 LOD 0 8  
12 LIT 0 1  
13 LIT 0 5  
14 OPR 0 22  
15 LIT 0 1  
16 OPR 0 3  
17 OPR 0 2  
18 OPR 0 19  
19 LOD 0 8  
20 LIT 0 1  
21 OPR 0 2  
22 STO 0 8  
23 JMP 0 3  
24 LIT 0 1  
25 STO 0 8  
26 LOD 0 8  
27 LIT 0 5  
28 OPR 0 12  
29 JPC 0 45  
30 LIT 0 3  
31 LOD 0 8  
32 LIT 0 1  
33 LIT 0 5  
34 OPR 0 22  
35 LIT 0 1  
36 OPR 0 3  
37 OPR 0 2  
38 OPR 0 18  
39 OPR 0 14  
40 LOD 0 8  
41 LIT 0 1  
42 OPR 0 2  
43 STO 0 8  
44 JMP 0 26  
45 RET
```

```
=== INTERPRETER OUTPUT ===  
10  
20  
30  
40  
50
```

D. Pengujian 4

Input

```
program TestKeywords;  
  
var  
  x, y: integer;  
  
begin  
  x := 10;  
  if x > 5 then  
    y := x * 2  
  else  
    y := x div 2;  
  writeln(y);  
  while y > 0 do  
    begin  
      y := y - 1;  
      writeln(y);  
    end;  
end.
```

Output

```
=== INTERMEDIATE CODE ===  
0 INT 0 5  
1 LIT 0 10  
2 STO 0 3  
3 LOD 0 3  
4 LIT 0 5  
5 OPR 0 11  
6 JPC 0 12  
7 LOD 0 3  
8 LIT 0 2  
9 OPR 0 4  
10 STO 0 4  
11 JMP 0 16  
12 LOD 0 3  
13 LIT 0 2  
14 OPR 0 5  
15 STO 0 4  
16 LOD 0 4  
17 OPR 0 14  
18 LOD 0 4
```

```
19 LIT 0 0
20 OPR 0 11
21 JPC 0 29
22 LOD 0 4
23 LIT 0 1
24 OPR 0 3
25 STO 0 4
26 LOD 0 4
27 OPR 0 14
28 JMP 0 18
29 RET

=== INTERPRETER OUTPUT ===
20
19
18
17
16
15
14
13
12
11
10
9
8
7
6
5
4
3
2
1
0
```

E. Pengujian 5

Input

```
program TestProc;

var
  i, total: integer;

procedure Add(x: integer);
begin
  total := total + x;
end;

begin
  total := 0;
  for i := 1 to 10 do
```

```
    Add(i);
    writeln('sum 1..10 = ', total);

    total := 100;
    for i := 5 downto 1 do
        Add(-i);
        writeln('after subtraction = ', total);
    end.
```

Output

```
=== INTERMEDIATE CODE ===
0 JMP 0 7
1 INT 0 4
2 LOD 1 4
3 LOD 0 3
4 OPR 0 2
5 STO 1 4
6 RET
7 INT 0 5
8 LIT 0 0
9 STO 0 4
10 LIT 0 1
11 STO 0 3
12 LOD 0 3
13 LIT 0 10
14 OPR 0 12
15 JPC 0 23
16 LOD 0 3
17 CAL 0 1
18 LOD 0 3
19 LIT 0 1
20 OPR 0 2
21 STO 0 3
22 JMP 0 12
23 LIT 0 'sum 1..10 = '
24 OPR 0 13
25 LOD 0 4
26 OPR 0 14
27 LIT 0 100
28 STO 0 4
29 LIT 0 5
30 STO 0 3
31 LOD 0 3
32 LIT 0 1
33 OPR 0 10
34 JPC 0 43
35 LOD 0 3
36 OPR 0 1
37 CAL 0 1
38 LOD 0 3
39 LIT 0 1
```

```
40 OPR 0 3
41 STO 0 3
42 JMP 0 31
43 LIT 0 'after subtraction = '
44 OPR 0 13
45 LOD 0 4
46 OPR 0 14
47 RET

=== INTERPRETER OUTPUT ===
sum 1..10 = 55
after subtraction = 85
```

F. Pengujian 6

Input

```
program M4Arith;

var
  a, b, c: integer;

begin
  a := 12;
  b := 5;
  c := (a + b) * 2 - 3;
  writeln('arith = ', c);
end.
```

Output

```
=== INTERMEDIATE CODE ===
0 INT 0 6
1 LIT 0 12
2 STO 0 3
3 LIT 0 5
4 STO 0 4
5 LOD 0 3
6 LOD 0 4
7 OPR 0 2
8 LIT 0 2
9 OPR 0 4
10 LIT 0 3
11 OPR 0 3
12 STO 0 5
13 LIT 0 'arith = '
14 OPR 0 13
15 LOD 0 5
16 OPR 0 14
17 RET

=== INTERPRETER OUTPUT ===
```

```
arith = 31
```

G. Pengujian 7

Input

```
program M4IfElse;

var
  x, y: integer;

begin
  x := 8;
  if x > 10 then
    y := x * 2
  else
    y := x div 2;
  writeln('ifelse = ', y);
end.
```

Output

```
=== INTERMEDIATE CODE ===
0 INT 0 5
1 LIT 0 8
2 STO 0 3
3 LOD 0 3
4 LIT 0 10
5 OPR 0 11
6 JPC 0 12
7 LOD 0 3
8 LIT 0 2
9 OPR 0 4
10 STO 0 4
11 JMP 0 16
12 LOD 0 3
13 LIT 0 2
14 OPR 0 5
15 STO 0 4
16 LIT 0 'ifelse = '
17 OPR 0 13
18 LOD 0 4
19 OPR 0 14
20 RET

=== INTERPRETER OUTPUT ===
ifelse = 4
```

H. Pengujian 8

Input

```
program M4WhileSum;

var
  i, sum: integer;

begin
  i := 1;
  sum := 0;
  while i <= 5 do
  begin
    sum := sum + i;
    i := i + 1;
  end;
  writeln('sum = ', sum);
end.
```

Output

```
=== INTERMEDIATE CODE ===
0 INT 0 5
1 LIT 0 1
2 STO 0 3
3 LIT 0 0
4 STO 0 4
5 LOD 0 3
6 LIT 0 5
7 OPR 0 12
8 JPC 0 18
9 LOD 0 4
10 LOD 0 3
11 OPR 0 2
12 STO 0 4
13 LOD 0 3
14 LIT 0 1
15 OPR 0 2
16 STO 0 3
17 JMP 0 5
18 LIT 0 'sum = '
19 OPR 0 13
20 LOD 0 4
21 OPR 0 14
22 RET

=== INTERPRETER OUTPUT ===
sum = 15
```

I. Pengujian 9

Input

```
program M4Func;
```



```
var
  n, result: integer;

function Factorial(k: integer): integer;
begin
  if k <= 1 then
    Factorial := 1
  else
    Factorial := k * Factorial(k - 1);
end;

function Square(x: integer): integer;
begin
  Square := x * x;
end;

begin
  n := 5;
  result := Factorial(n);
  writeln('fact(', n, ') = ', result);
  writeln('square(', n, ') = ', Square(n));
end.
```

Output

```
=== INTERMEDIATE CODE ===
0 JMP 0 23
1 INT 0 4
2 LOD 0 3
3 LIT 0 1
4 OPR 0 12
5 JPC 0 9
6 LIT 0 1
7 STO 0 0
8 JMP 0 16
9 LOD 0 3
10 LOD 0 3
11 LIT 0 1
12 OPR 0 3
13 CAL 1 1
14 OPR 0 4
15 STO 0 0
16 RET
17 INT 0 4
18 LOD 0 3
19 LOD 0 3
20 OPR 0 4
21 STO 0 0
22 RET
23 INT 0 5
24 LIT 0 5
25 STO 0 3
```

```
26 LOD 0 3
27 CAL 0 1
28 STO 0 4
29 LIT 0 'fact('
30 OPR 0 13
31 LOD 0 3
32 OPR 0 13
33 LIT 0 ')' = '
34 OPR 0 13
35 LOD 0 4
36 OPR 0 14
37 LIT 0 'square('
38 OPR 0 13
39 LOD 0 3
40 OPR 0 13
41 LIT 0 ')' = '
42 OPR 0 13
43 LOD 0 3
44 CAL 0 17
45 OPR 0 14
46 RET

=== INTERPRETER OUTPUT ===
fact(5) = 120
square(5) = 25
```

J. Pengujian 10

Input

```
program M4ProcFunc;

var
  n, total: integer;

procedure AddTwice(x: integer);
begin
  total := total + x;
  total := total + x;
end;

function Square(x: integer): integer;
begin
  Square := x * x;
end;

begin
  total := 0;
  n := 3;
  AddTwice(n);
  total := total + Square(n);
  writeln('procfunc = ', total);
```

```
end.
```

Output

```
=== INTERMEDIATE CODE ===
0 JMP 0 17
1 INT 0 4
2 LOD 1 4
3 LOD 0 3
4 OPR 0 2
5 STO 1 4
6 LOD 1 4
7 LOD 0 3
8 OPR 0 2
9 STO 1 4
10 RET
11 INT 0 4
12 LOD 0 3
13 LOD 0 3
14 OPR 0 4
15 STO 0 0
16 RET
17 INT 0 5
18 LIT 0 0
19 STO 0 4
20 LIT 0 3
21 STO 0 3
22 LOD 0 3
23 CAL 0 1
24 LOD 0 4
25 LOD 0 3
26 CAL 0 11
27 OPR 0 2
28 STO 0 4
29 LIT 0 'procfunc = '
30 OPR 0 13
31 LOD 0 4
32 OPR 0 14
33 RET

=== INTERPRETER OUTPUT ===
procfunc = 15
```

K. Pengujian 11

Input

```
program TestDivZero;

var
  a, b: integer;
```

```
begin
  a := 10;
  b := 0;
  writeln(a div b);
end.
```

Output

```
=== INTERMEDIATE CODE ===
0 INT 0 5
1 LIT 0 10
2 STO 0 3
3 LIT 0 0
4 STO 0 4
5 LOD 0 3
6 LOD 0 4
7 OPR 0 5
8 OPR 0 14
9 RET

=== INTERPRETER OUTPUT ===

=== RUNTIME ERRORS ===
Division by zero
```

L. Pengujian 12

Input

```
program InfiniteLoop;

var
  x: integer;

begin
  x := 1;
  while x > 0 do
  begin
    x := x + 1;
  end;
end.
```

Output

```
=== INTERMEDIATE CODE ===
0 INT 0 4
1 LIT 0 1
2 STO 0 3
3 LOD 0 3
4 LIT 0 0
5 OPR 0 11
```

```
6 JPC 0 12
7 LOD 0 3
8 LIT 0 1
9 OPR 0 2
10 STO 0 3
11 JMP 0 3
12 RET

=== INTERPRETER OUTPUT ===

=== RUNTIME ERRORS ===
Execution step limit exceeded: possible infinite loop
```

M. Pengujian 13

Input

```
program TestOutOfBounds;

var
  arr: array[1..5] of integer;
  i: integer;

begin
  arr[1] := 10;
  arr[2] := 20;
  arr[3] := 30;
  arr[4] := 40;
  arr[5] := 50;
  writeln('Array[3] = ', arr[3]);
  i := arr[10];
  writeln('Out of bounds access result: ', i);
end.
```

Output

```
=== INTERMEDIATE CODE ===
0 INT 0 9
1 LIT 0 10
2 LIT 0 3
3 LIT 0 1
4 LIT 0 1
5 LIT 0 5
6 OPR 0 22
7 LIT 0 1
8 OPR 0 3
9 OPR 0 2
10 OPR 0 19
11 LIT 0 20
12 LIT 0 3
13 LIT 0 2
14 LIT 0 1
```

```
15 LIT 0 5
16 OPR 0 22
17 LIT 0 1
18 OPR 0 3
19 OPR 0 2
20 OPR 0 19
21 LIT 0 30
22 LIT 0 3
23 LIT 0 3
24 LIT 0 1
25 LIT 0 5
26 OPR 0 22
27 LIT 0 1
28 OPR 0 3
29 OPR 0 2
30 OPR 0 19
31 LIT 0 40
32 LIT 0 3
33 LIT 0 4
34 LIT 0 1
35 LIT 0 5
36 OPR 0 22
37 LIT 0 1
38 OPR 0 3
39 OPR 0 2
40 OPR 0 19
41 LIT 0 50
42 LIT 0 3
43 LIT 0 5
44 LIT 0 1
45 LIT 0 5
46 OPR 0 22
47 LIT 0 1
48 OPR 0 3
49 OPR 0 2
50 OPR 0 19
51 LIT 0 'Array[3] = '
52 OPR 0 13
53 LIT 0 3
54 LIT 0 3
55 LIT 0 1
56 LIT 0 5
57 OPR 0 22
58 LIT 0 1
59 OPR 0 3
60 OPR 0 2
61 OPR 0 18
62 OPR 0 14
63 LIT 0 3
64 LIT 0 10
65 LIT 0 1
66 LIT 0 5
67 OPR 0 22
```

```
68 LIT 0 1
69 OPR 0 3
70 OPR 0 2
71 OPR 0 18
72 STO 0 8
73 LIT 0 'Out of bounds access result: '
74 OPR 0 13
75 LOD 0 8
76 OPR 0 14
77 RET

=== INTERPRETER OUTPUT ===
Array[3] = 30

=== RUNTIME ERRORS ===
Array index out of bounds: 10 not in [1..5]
```

N. Pengujian 14

Input

```
program TestOverflow;

var
  a, b: integer;

begin
  a := 2000000000;
  b := a + a;
  writeln(b);
end.
```

Output

```
=== INTERMEDIATE CODE ===
0 INT 0 5
1 LIT 0 2000000000
2 STO 0 3
3 LOD 0 3
4 LOD 0 3
5 OPR 0 2
6 STO 0 4
7 LOD 0 4
8 OPR 0 14
9 RET

=== INTERPRETER OUTPUT ===

=== RUNTIME ERRORS ===
Numerical overflow/underflow in arithmetic: 2000000000 op 2000000000 =
4000000000
```

O. Pengujian 15

Input

```
program TestRecursion;  
  
var  
  n: integer;  
  
function Inf(k: integer): integer;  
begin  
  Inf := Inf(k + 1);  
end;  
  
begin  
  n := Inf(0);  
  writeln(n);  
end.
```

Output

```
=== INTERMEDIATE CODE ===  
0 JMP 0 8  
1 INT 0 4  
2 LOD 0 3  
3 LIT 0 1  
4 OPR 0 2  
5 CAL 1 1  
6 STO 0 0  
7 RET  
8 INT 0 4  
9 LIT 0 0  
10 CAL 0 1  
11 STO 0 3  
12 LOD 0 3  
13 OPR 0 14  
14 RET  
  
=== INTERPRETER OUTPUT ===  
  
=== RUNTIME ERRORS ===  
Stack overflow: maximum recursion depth (1000) exceeded
```


IV. Kesimpulan & Saran

A. Kesimpulan

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, intermediate code generator dan interpreter pada compiler berhasil dikembangkan menggunakan pendekatan Three-Address Code berbasis Stack Machine. Sistem mampu menurunkan Decorated AST dan Symbol Table menjadi daftar instruksi (ICInstr) serta mengeksekusinya melalui siklus fetch-decode-execute dengan baik. Selain itu, arsitektur interpreter yang dibangun telah dilengkapi dengan pengamanan memori untuk mendeteksi kerentanan eksekusi (vulnerabilities) seperti invalid jump, stack overflow dan underflow, division by zero, serta akses variabel di luar batas (out-of-bounds). Hasil pengujian menunjukkan bahwa program dapat memproses berbagai fitur dengan tepat, mulai dari operasi aritmatika, percabangan (if/case), perulangan (while/repeat/for), hingga pemanggilan prosedur dan fungsi yang melibatkan manajemen stack frame. Dengan demikian, milestone ini berhasil memenuhi tujuan utama dalam menyintesis Intermediate Code menggunakan strategi patching label dan mengeksekusinya secara konsisten, aman, dan efisien.

B. Saran

Untuk pengembangan selanjutnya, intermediate code generator dan interpreter dapat ditingkatkan dengan mengimplementasikan modul runtime type checking secara menyeluruh agar compiler dapat mendeteksi deklarasi tipe data implisit atau perubahan tipe data saat eksekusi (runtime) dengan lebih presisi. Selain itu, mekanisme perlindungan terhadap kerentanan (vulnerabilities) dapat diperluas untuk memberikan pesan exception atau runtime error yang lebih spesifik dan informatif, sehingga mencegah kerusakan berlebih dan memudahkan programmer dalam perbaikan kode.

Referensi

"Compilers - Principles, Techniques, and Tools (2006)"

[https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf)

Guidebook Konvensi Intermediate Code

☰ Lampiran Spesifikasi Milestone 4 - Tubes IF2224 TBFO - Guidebook Konvensi In...

Lampiran

Github : [BST-Tubes-IF2224-2026](#)

Pembagian Tugas :

NIM	Persentase
13524019	25%
13524033	25%
13524043	25%
13524101	25%